Technical Report: Network Defense Firewall and Cryptographic Analysis

Author: Le Chi Bach

Table of Contents

Introduction 4

Section 1: Intrusion Detection 5

Q1.1 5

Q1.2 5

Q1.3 5

Q1.4 6

Q1.5 6

Q1.6 6

Q1.7 7

Q1.8 9

Q1.9 10

Section 2: Firewalls 11

Q2.1 11

Q2.2 12

Q2.3 13

Q2.4 14

Q2.5 16

Section 3: Proper Usage of Symmetric Encryption 18

Q3.1 18

Q3.2 18

Q3.3 20

Section 4: Penetration Testing 22

Q4.1 22

Q4.2 23

Q4.3 24

Q4.4 25

Q4.5 25
Q4.6 26
Table of Figures 27
Reference 29

Introduction

This report provides a comprehensive security assessment of the target system by evaluating both defensive configurations and offensive testing methodologies. The scope of this evaluation covers intrusion detection methods and firewall infrastructure to analyze traffic filtering and threat detection capabilities using iptables and Snort. Furthermore, the assessment examines cryptographic implementation by comparing the structural resilience of symmetric encryption modes, specifically Electronic Codebook (ECB) and Cipher Block Chaining (CBC). Finally, active penetration testing was conducted using Nmap for network reconnaissance and port auditing, alongside Flawfinder for static application security testing (SAST) to identify source code vulnerabilities.

Section 1: Intrusion Detection

In this section, recommendations on intrusion detection methods (Network Intrusion Detection System (NIDS), Host-based Intrusion Detection System (HIDS), or Logging) will be given based on the given scenarios.

Q1.1

STCorp is hosting a web application over HTTPS and needs to detect any use of blacklisted characters in real time.

Solution: HIDS. This scenario requires detection of backlisted characters, which means that the detection method has to work with encrypted messages. HIDS resides on the host where data is decrypted and can provide real time monitoring.

Q1.2

STCorp is hosting a web application over HTTP and wants to pass all user traffic through an anomaly detection algorithm (which uses some computationally expensive Machine Learning). The web application needs to have low latency when many users are online during the day.

Solution: NIDS. NIDS is the most optimal method because it is installed in the network, therefore it does not take resources from the web application, keeping it low latency for users. NIDS is cheap and provide linear scaling, so in case traffic surges, computational power can be added without over-provisioning. Since the application is hosted over HTTP (the data is not encrypted), NIDS can examine the data in the traffic.

Q1.3

STCorp uses the Simple Mail Transfer Protocol (SMTP) for email and wants to be able to quickly detect phishing attacks against any of their internal computers. SMTP runs on port 25 and is unencrypted.

Solution: NIDS. Because SMTP is unencrypted, NIDS can directly inspect the payload for malicious content. NIDS resides on the network, monitoring all devices within a subnet, therefore it is suitable to detect phishing attacks against several computers. NIDS provides real time monitoring for quick detection.

From Q1.1 to Q1.6, The most optimal intrusion detection technique (signature-based, anomaly-based, specification-based, behavioral, etc.) will be selected based on given scenarios.

Q1.4

STCorp wants to detect script kiddies (hackers who primarily use publicly available tools or exploits).

Solution: Signature-based. Script kiddies use publicly available tools, which can also be previously added to a database of known threats patterns. The patterns will be compared with real-time events to detect commonly known attacks performed by script kiddies.

Q1.5

STCorp wants to detect any attempts by their employees to access the protected /etc/passwd file.

Solution: Specification-based. Keep a whitelist of allowed action among employees, not including accessing /etc/passwd, so that any attempt of accessing the file will be flagged.

Q1.6

STCorp wants to detect a seasoned l33t h4x0r who crafts custom exploits for each attack.

Solution: Anomaly based. Use machine learning to build a model normal behavior then match it against malicious activity in the belief of any attack would appear abnormally.

Other possible solution(s): Specification based. Like anomaly based, but it specifies normal behavior instead of building a model.

Question Q1.7 to Q1.9 will be performed in a Linux Virtual Machine in the scenario of STCorp wanting to use Snort to enable IDS to monitor and alert ping test in subnet (10.0.2.0/24). Because there were differences in the configuration in the requirements and the SEED lab in which the exercised would be done, Snort will be implemented in subnet 192.168.60.0/24.

Q1.7

The first step of this exercise is to build the container (using "docker-compose build") and start all the containers (using "docker-compose up -d").

```
[08/16/26]seed@VM:~/.../Labsetup$ docker-compose build
HostA uses an image, skipping
Host1 uses an image, skipping
Host2 uses an image, skipping
Host3 uses an image, skipping
Building Router
Step 1/2 : FROM handsonsecurity/seed-ubuntu:large
 ---> cecb04fbf1dd
Step 2/2 : RUN apt-get update     && apt-get install -y kmod      && apt-get clean
 ---> Using cache
 ---> feb31e62a166

Successfully built feb31e62a166
Successfully tagged seed-router-image:latest
[08/16/26]seed@VM:~/.../Labsetup$ docker-compose up -d
Creating network "net-10.9.0.0" with the default driver
Creating network "net-192.168.60.0" with the default driver
Creating seed-router        ... done
Creating host1-192.168.60.5 ... done
Creating host2-192.168.60.6 ... done
Creating hostA-10.9.0.5     ... done
Creating host3-192.168.60.7 ... done
```

Figure 1: Building and starting containers.

Next, the containers' names and IDs can be acquired with the command (dockps). The snort rule will be configured inside the seed-router container to detect external messages coming to the internal subnet.

```
[08/16/26]seed@VM:~/.../Labsetup$ dockps
480be90a3804  seed-router
b168a8aabb9e  host3-192.168.60.7
467e04e8e08f  host1-192.168.60.5
507f9dd1386b  hostA-10.9.0.5
5a02d4e38f78  host2-192.168.60.6
[08/16/26]seed@VM:~/.../Labsetup$
```

Figure 2: All containers and their IDs.

Router container was entered using docksh + container's ID. After that, snort installation is confirmed by checking its version.

```
[08/16/26]seed@VM:~/.../Labsetup$ docksh 48
root@480be90a3804:/# snort -V

   ,,_     -*> Snort! <*-
  o"  )~   Version 2.9.7.0 GRE (Build 149)
   ''''    By Martin Roesch & The Snort Team: http://www.
snort.org/contact#team
           Copyright (C) 2014 Cisco and/or its affiliates
. All rights reserved.
           Copyright (C) 1998-2013 Sourcefire, Inc., et a
l.
           Using libpcap version 1.9.1 (with TPACKET_V3)
           Using PCRE version: 8.39 2016-06-14
           Using ZLIB version: 1.2.11

root@480be90a3804:/#
```

Figure 3: Checking Snort installation inside router container.

Snort rules will then be added to /etc/snort/rules/local.rules[1]. The rule used for this question is shown as bellow:

"alert icmp 10.9.0.0/24 any -> 192.168.60.0/24 any (msg:"Le Chi Bach external ICMP test detected against the subnet"; sid:9999999; rev:1)"

- *"alert"*: specified action[2].
- *"icmp"*: specified protocol[2] (ICMP is the protocol that ping uses[3]).
- *"10.9.0.0/24 any"*: source IP addresses[2] (in this case the external host has the IP address of 10.9.0.5).
- *"->"*: Direction operator (the addresses on the left side of this operator is the source, on the right sideis the destination)[2].
- *"192.168.60.0/24 any"*: destination IP addresses[2].
- *"msg:"Le Chi Bach external ICMP test detected against the subnet""*: message that will be outputted in the alert[2].
- *"sid:9999999"*: optional ID for quick identification.[2] It should be set starting from 1000000 onward because values from 0-999999 are used in rules included with Snort distribution[2].
- *"rev: 1"*: specified revision number of a Snort rule and should be incremented by one each time a change is made to a rule[2].
-

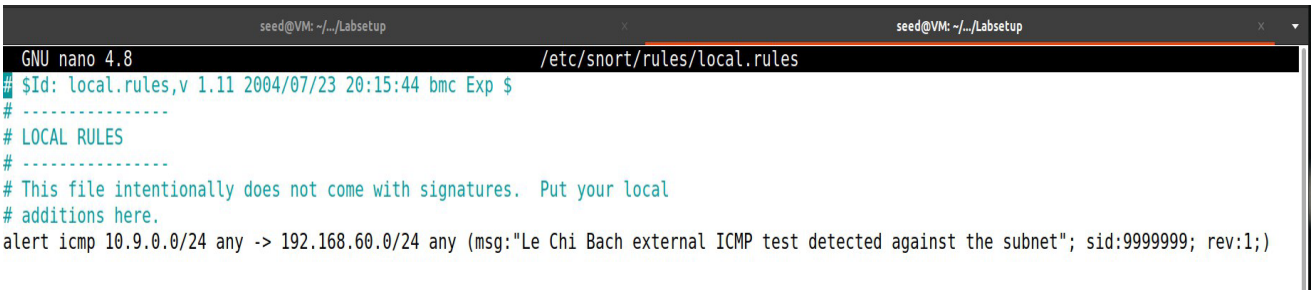


Figure 3: Snort rule set in .etc.snort/rules/local.rules.

Q1.8

To start detecting using snort, the command bellow was used in router console:

"snort -A console -q -c /etc/snort/snort.conf -i eth0"

- *-A console*: send alert to console screen[4].
- *-q*: diabase banner and status report[4] to make the output appear cleaner.

- *-c /etc/snort/snort.conf*: apply rules configured in snort.conf[4], which has a link to /etc/snort/rules/local.rules(the location of the snort rule set in Q1.7).
- *-i eth0*: Snort will listen on eth0.

After running the command above, the subnet broadcast IP address, which is 192.168.60.255, will be ping from host 10.9.0.5.

The result is shown in Figure 5, as the router console output the message specified in local.rules. The source and destination IP addresses matched the specifications; therefore, the alerts in Figure 5 shows the success of the Snort implementing attempt.

```
root@507f9dd1386b:/# ping -b 192.168.60.255
PING 192.168.60.255 (192.168.60.255) 56(84) bytes of data.
^C
--- 192.168.60.255 ping statistics ---
32 packets transmitted, 0 received, 100% packet loss, time 31781ms
```

Figure 4: Ping subnet's broadcast IP address from 10.9.0.5.

```
oot@480be90a3804:/# snort -A console -q -c /etc/snort/snort.conf -i eth0
8/19-08:00:21.962644  [**] [1:366:7] ICMP PING *NIX [**] [Classification: Misc activity] [Priority: 3] {ICMP} 10.9.0.
 -> 192.168.60.255
8/19-08:00:21.962644  [**] [1:9999999:1] Le Chi Bach external ICMP test detected against the subnet [**] [Priority: 0
 {ICMP} 10.9.0.5 -> 192.168.60.255
8/19-08:00:21.962644  [**] [1:384:5] ICMP PING [**] [Classification: Misc activity] [Priority: 3] {ICMP} 10.9.0.5 ->
92.168.60.255
8/19-08:00:22.991671  [**] [1:366:7] ICMP PING *NIX [**] [Classification: Misc activity] [Priority: 3] {ICMP} 10.9.0.
 -> 192.168.60.255
8/19-08:00:22.991671  [**] [1:9999999:1] Le Chi Bach external ICMP test detected against the subnet [**] [Priority: 0
 {ICMP} 10.9.0.5 -> 192.168.60.255
8/19-08:00:22.991671  [**] [1:384:5] ICMP PING [**] [Classification: Misc activity] [Priority: 3] {ICMP} 10.9.0.5 ->
92.168.60.255
8/19-08:00:24.029260  [**] [1:366:7] ICMP PING *NIX [**] [Classification: Misc activity] [Priority: 3] {ICMP} 10.9.0.
 -> 192.168.60.255
8/19-08:00:24.029260  [**] [1:9999999:1] Le Chi Bach external ICMP test detected against the subnet [**] [Priority: 0
 {ICMP} 10.9.0.5 -> 192.168.60.255
8/19-08:00:24.029260  [**] [1:384:5] ICMP PING [**] [Classification: Misc activity] [Priority: 3] {ICMP} 10.9.0.5 ->
```

Figure 5: Snort alerts seen in router's console in Q1.8.

Q1.9

In this question, IP1 will be ping from within the subnet using host 192.168.60.5 (as shown in Figure 6). The result in Figure 7 showed that Snort was able to alert the ping message going from within the subnet to an external IP.

```
root@467e04e8e08f:/# ping 3.4.1.9
PING 3.4.1.9 (3.4.1.9) 56(84) bytes of data.
^C
--- 3.4.1.9 ping statistics ---
7 packets transmitted, 0 received, 100% packet loss, time 6169ms
```

Figure 6: Ping IP1 from internal host 192.168.60.5.

```
root@480be90a3804:/# snort -A console -q -c /etc/snort/snort.conf -i eth0
08/19-15:38:19.760027  [**] [1:366:7] ICMP PING *NIX [**] [Classification: Misc activity] [Priority: 3] {ICMP} 192.168
.60.5 -> 3.4.1.9
08/19-15:38:19.760027  [**] [1:384:5] ICMP PING [**] [Classification: Misc activity] [Priority: 3] {ICMP} 192.168.60.5
 -> 3.4.1.9
08/19-15:38:20.802636  [**] [1:366:7] ICMP PING *NIX [**] [Classification: Misc activity] [Priority: 3] {ICMP} 192.168
.60.5 -> 3.4.1.9
08/19-15:38:20.802636  [**] [1:384:5] ICMP PING [**] [Classification: Misc activity] [Priority: 3] {ICMP} 192.168.60.5
 -> 3.4.1.9
08/19-15:38:21.812813  [**] [1:366:7] ICMP PING *NIX [**] [Classification: Misc activity] [Priority: 3] {ICMP} 192.168
.60.5 -> 3.4.1.9
08/19-15:38:21.812813  [**] [1:384:5] ICMP PING [**] [Classification: Misc activity] [Priority: 3] {ICMP} 192.168.60.5
 -> 3.4.1.9
08/19-15:38:22.835364  [**] [1:366:7] ICMP PING *NIX [**] [Classification: Misc activity] [Priority: 3] {ICMP} 192.168
.60.5 -> 3.4.1.9
08/19-15:38:22.835364  [**] [1:384:5] ICMP PING [**] [Classification: Misc activity] [Priority: 3] {ICMP} 192.168.60.5
 -> 3.4.1.9
```

Figure 7: Snort alerts seen in router's console in Q1.9.

Section 2: Firewalls

In Section 2, iptables – a Linux pre-built firewall will be used to protect a given network. Firewall rules will be implemented based on the requirement of each question in this section. The exercises were performed in a Linux virtual machine.

Q2.1

In this question, a stateless firewall rule will be set up to prevent routers pinging an outside machine with IP1. Below shows the command used to set the rule:

"iptables -A OUTPUT -p icmp --icmp-type echo-request -s 3.4.1.9 -j DROP"

- *-A OUTPUT*: Append the rule to the OUTPUT chain[5].
- *-p icmp*: filter the packet with ICMP rule[5] (ping uses ICMP protocol).
- *--icmp-type echo-request*: Set ICMP condition to drop request type[6].
- *-s 3.4.1.9*: Specify source[5].
- *-j DROP*: Specify the target of the rule. If any packet fits the description in the rule, it will be dropped[5].

After running the command in the router console, IP1 was ping. The result is shown in Figure 8. When the router tried to ping IP1, there is a message outputted saying "Operation is not permitted.", indicating the success of the firewall rule.

```
root@235b04e85c0c:/# iptables -A OUTPUT -p icmp --icmp-type echo-request -d 3.4.1.9 -j DROP
root@235b04e85c0c:/# ping 3.4.1.9
PING 3.4.1.9 (3.4.1.9) 56(84) bytes of data.
ping: sendmsg: Operation not permitted
ping: sendmsg: Operation not permitted
ping: sendmsg: Operation not permitted
ping: sendmsg: Operation not permitted
ping: sendmsg: Operation not permitted
ping: sendmsg: Operation not permitted
ping: sendmsg: Operation not permitted
ping: sendmsg: Operation not permitted
ping: sendmsg: Operation not permitted
ping: sendmsg: Operation not permitted
^C
--- 3.4.1.9 ping statistics ---
10 packets transmitted, 0 received, 100% packet loss, time 9210ms
```

Figure 8: Result of firewall rule in Q2.1

Q2.2

In this question, a stateless firewall rule will be set up to prevent 8.8.8.8 to echo back a ping request from the router. Below shows the command used to set the rule:

"iptables -A INPUT -p icmp --icmp-type echo-reply -s 8.8.8.8 -j DROP"

- *-A INPUT*: Append the rule to the INPUT chain [5].
- *-p icmp*: filter the packet with ICMP rule[5].
- *--icmp-type echo-reply*: Set ICMP condition to drop reply type[6].
- *-s 8.8.8.8*: Specify source[5].
- *-j DROP*: Specify the target of the rule. If any packet fits the description in the rule, it will be dropped[5].

After running the command, 8.8.8.8 was ping from the router. The results in Figure 9 showed that there was no response despite 23 packets being sent.

I used "iptables -L INPUT" to see all listed rules. It is observed that 23 packets (equivalent with 23 packets sent by router) from 8.8.8.8 was dropped. The firewall had stopped the replying packets from the indicated destination and the implementation was successful.

```
root@235b04e85c0c:/# iptables -A INPUT -p icmp --icmp-type echo-re
ply -s 8.8.8.8 -j DROP
root@235b04e85c0c:/# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
^C
--- 8.8.8.8 ping statistics ---
23 packets transmitted, 0 received, 100% packet loss, time 22553ms

root@235b04e85c0c:/# iptables -L INPUT -v -n
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source
  destination
   23  1932 DROP       icmp --  *      *       8.8.8.8
  0.0.0.0/0            icmptype 0
```

Figure 9: Result of firewall rule in Q2.2.

Q2.3

In this question, a stateless firewall rule will be set up to:

1) Allow all internal hosts in the subnet run a telnet session.
2) Allow outside hosts to only be able to access the telnet server on host 192.168.60.5.
3) Deny all other packets.

To achieve that, the following commands were used:

```
(1) 5b04e85c0c:/# iptables -A FORWARD -p tcp -s 192.168.60.0/24 -o eth0 --dport 23 -j ACCEPT
(2) 5b04e85c0c:/# iptables -A FORWARD -p tcp -i eth0 --sport 23 -d 192.168.60.0/24 -j ACCEPT
(3) 5b04e85c0c:/# iptables -A FORWARD -p tcp -i eth0 -d 192.168.60.5 --dport 23 -j ACCEPT
(4) 5b04e85c0c:/# iptables -A FORWARD -p tcp -o eth0 -s 192.168.60.5 --sport 23 -j ACCEPT
(5) 5b04e85c0c:/# iptables -A FORWARD -j DROP
```

Figure 10: Firewall rules set up for Q2.3.

The (1) rule was used to allow inner subnet to telnet to the outside via eth0 interface. The (2) rule was used to allow outsider hosts to telnet into the subnet via eth0 interface. The (3) rule was used to allow outsider hosts to telnet only to host 192.168.60.5 via eth0. The (4) rule was used to allow host 192.168.60.5 to telnet to outsiders via eth0. The (5) rule drops all the packets that do not meet the above condition.

After setting the rule, host 10.9.0.5 from inside the subnet (Figure 11) and vice versa (Figure 12). The result showed that the telnet session was established at both times, which satisfied requirements 1 and 2.

```
root@7531cf8f7cc3:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
61da7c7a0fde login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)
```

Figure 11: Telnet from 192.168.0.5 to 10.9.0.5 in Q2.3.

```
root@61da7c7a0fde:/# telnet 192.168.60.5
Trying 192.168.60.5...
Connected to 192.168.60.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
7531cf8f7cc3 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)
```

Figure 12: Telnet from 10.9.0.5 to host 192.168.60.5 in Q2.3.

Next, an attempt to ping to 192.168.60.6 from 10.9.0.5 was made (Figure 13). It took too long to establish a session indicating that the attempt was a fail. Therefore, the third requirement of this question is satisfied.

```
root@61da7c7a0fde:/# telnet 192.168.60.6
Trying 192.168.60.6...
```

Figure 13: Telnet from 10.9.0.5 to host 192.168.60.6 in Q2.3.

Q2.4

In this question, a stateful firewall rule will be set up to:

1) Block all hosts in the subnet to run a telnet session connecting to outsiders

2) Allow outside hosts to only be able to access the telnet server on host 192.168.60.5.
3) Deny all other packets.

To achieve that, the following commands were used:

```
(1) @235b04e85c0c:/# iptables -A FORWARD -m conntrack --cts
tate ESTABLISHED,RELATED -j ACCEPT
(2) @235b04e85c0c:/# iptables -A FORWARD -p tcp -s 192.168.
60.0/24 -o eth0 --dport 23 -m conntrack --ctstate NEW -j DR
OP
(3) @235b04e85c0c:/# iptables -A FORWARD -p tcp -i eth0 -d
192.168.60.5 --dport 23 -m conntrack --ctstate NEW -j ACCEP
T
(4) @235b04e85c0c:/# iptables -A FORWARD -j DROP
root@235b04e85c0c:/#
```

Figure 14: Firewall rules set up for Q2.4.

The (1) rule was used to allow a response from or related to an established connection. The (2) rule was used to block internal hosts to telnet outside the subnet. The (3) rule was used to allow outsider hosts to telnet only to host 192.168.60.5 via eth0. The (4) rule drops all the packets that do not meet the above condition. The rules above check the states of the packets (ESTABLISHED, RELATED, NEW), which make them stateful.

After setting the rules, 10.9.0.5 was telnet from internal host 192.168.60.5 (Figure 15) and vice versa (Figure 16). 10.9.0.5 successfully established a telnet connection with 192.168.60.5, while the internal host could not do the same, which means requirements 1 and 2 are satisfied. An attempt to telnet host192.168.60.6 from 10.9.0.5 had also been made (Figure 17), but because it had not been permitted in the firewall rules due to requirement 3, the attempt was not successful.

```
root@7531cf8f7cc3:/# telnet 10.9.0.5
Trying 10.9.0.5...
```

Figure 15: Telnet from 192.168.0.5 to 10.9.0.5 in Q2.4.

```
root@61da7c7a0fde:/# telnet 192.168.60.5
Trying 192.168.60.5...
Connected to 192.168.60.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
7531cf8f7cc3 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)
```

Figure 16: Telnet from 10.9.0.5 to host 192.168.60.5 in Q2.4.

```
root@61da7c7a0fde:/# telnet 192.168.60.6
Trying 192.168.60.6...
```

Figure 17: Telnet from 10.9.0.5 to host 192.168.60.6 in Q2.4.

Q2.5

In this question, a stateful firewall rule will be set up to:

1) Allow all hosts in the subnet to run a telnet session connecting to outsiders
2) Allow outside hosts to only be able to access the telnet server on host 192.168.60.5.
3) Deny all other packets.

To achieve that, the following commands were used:

```
(1) @235b04e85c0c:/# iptables -A FORWARD -m conntrack --ctstate EST
ABLISHED,RELATED -j ACCEPT
(2) @235b04e85c0c:/# iptables -A FORWARD -p tcp -s 192.168.60.0/24
-o eth0 --dport 23 -m conntrack --ctstate NEW -j ACCEPT
(3) @235b04e85c0c:/# iptables -A FORWARD -p tcp -i eth0 -d 192.168.
60.5 --dport 23 -m conntrack --ctstate NEW -j ACCEPT
(4) @235b04e85c0c:/# iptables -A FORWARD -j DROP
```

Figure 18: Firewall rules set up for Q2.5.

The (1) rule was used to allow a response from or related to an established connection. The (2) rule was used to allow internal hosts to telnet outside the subnet. The (3) rule was used to allow outsider hosts to telnet only to host 192.168.60.5 via eth0. The (4) rule drops all the packets that do not meet the above condition.

After setting the rules, 10.9.0.5 was telnet from internal host 192.168.60.5 (Figure 19) and vice versa (Figure 20). Both attempts were successful, which means requirements 1 and 2 are satisfied. An attempt to telnet host 192.168.60.6 from 10.9.0.5 had also been made (Figure 21), but because it had not been permitted in the firewall rules due to requirement 3, the attempt was not successful.

```
root@7531cf8f7cc3:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
61da7c7a0fde login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)
```

Figure 19: Telnet from 192.168.0.5 to 10.9.0.5 in Q2.5.

```
root@61da7c7a0fde:/# telnet 192.168.60.5
Trying 192.168.60.5...
Connected to 192.168.60.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
7531cf8f7cc3 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)
```

Figure 20: Telnet from 10.9.0.5 to host 192.168.60.5 in Q2.5.

```
root@61da7c7a0fde:/# telnet 192.168.60.6
Trying 192.168.60.6...
```

Figure 21: Telnet from 10.9.0.5 to host 192.168.60.6 in Q2.5.

Section 3: Proper Usage of Symmetric Encryption

Q3.1

To encrypt pic_original.bmp using AES ECB, the command in Figure 22 was used.

```
[08/23/26]seed@VM:~$ openssl enc -aes-128-ecb -e -in pic_original.b
mp -out pic_ecb.bmp -K aa44112233441199aa44112233441199
```

Figure 22: Encrypting pic_original.bmp using AES ECB.

To encrypt pic_original.bmp using AES CBC, the command in Figure 23 was used.

```
[08/23/26]seed@VM:~$ openssl enc -aes-128-cbc -e -in pic_original.b
mp -out pic_cbc.bmp -K aa44112233441199aa44112233441199 -iv 0a04010
2030401090a04010203040109
```

Figure 23: Encrypting pic_original.bmp using AES CBC.

The result is shown in Figure 24.

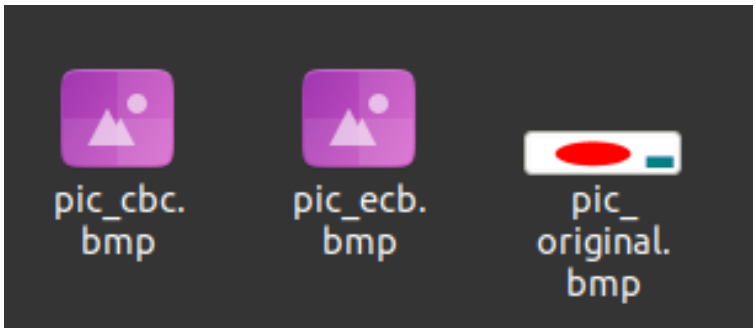


Figure 24: Encrypted pictures.

Q3.2

To see the picture, the header of the encrypted picture must be set correctly. To do that, the original pic_original.bmp was opened in GHex and the first 54 bytes (containing the picture's information header) was obtained. The 2 encrypted pictures were also opened with GHex and had their first 54 bytes replaced with those from the original. With that being done, the 2 encrypted pictures can be observed.

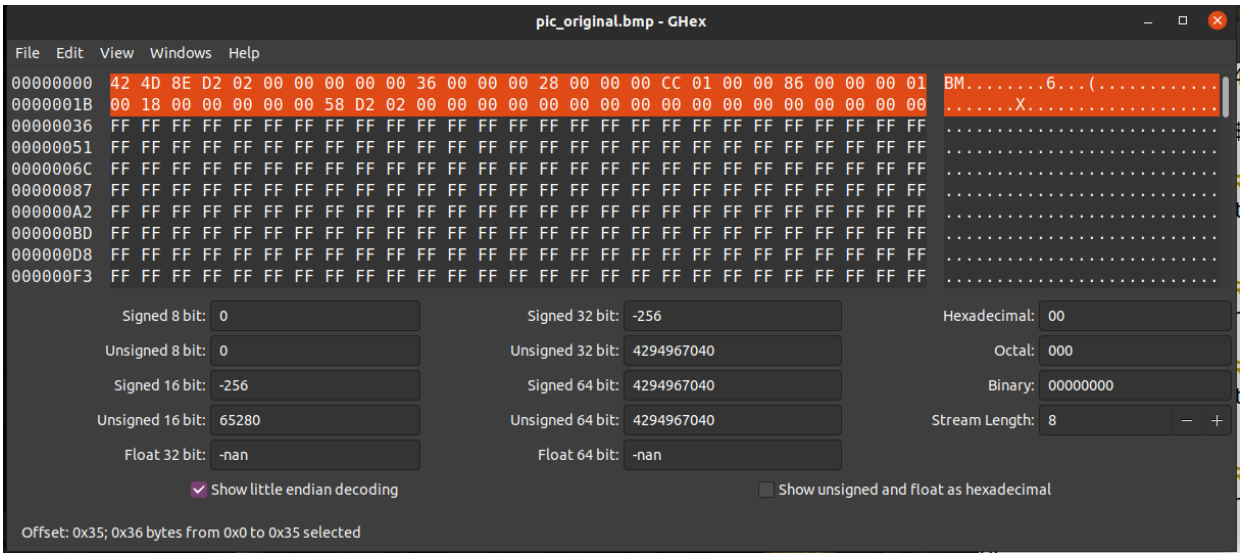


Figure 25: Extracting header information from pic_original.bmp using GHex.

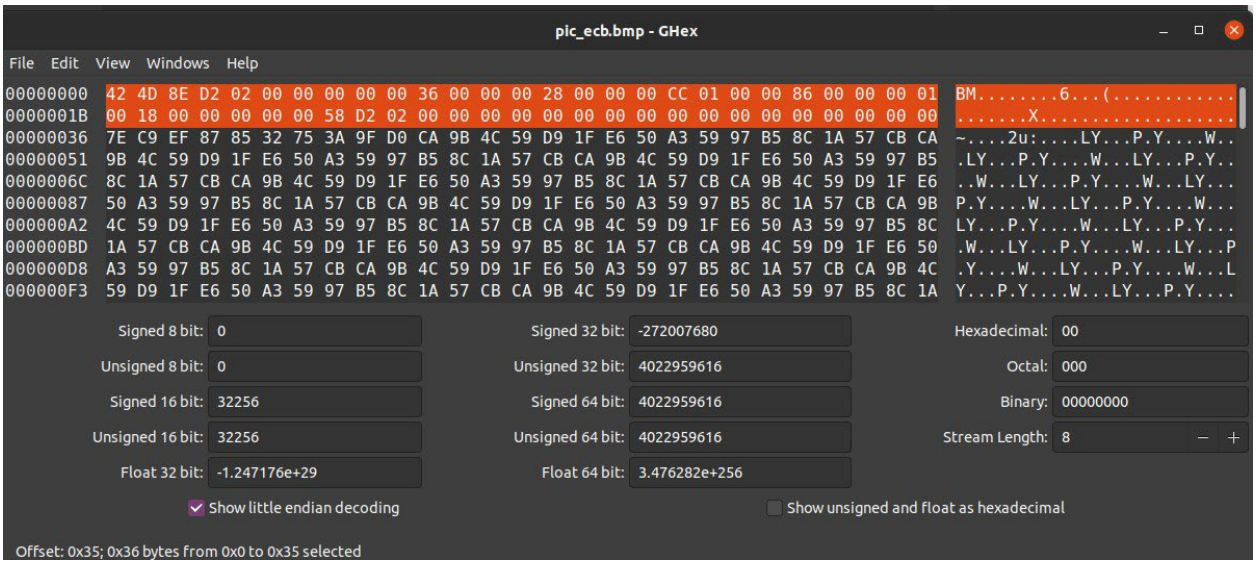


Figure 26: Replacing first 54 bytes with information header in pic_ecb.bmp using Ghex.

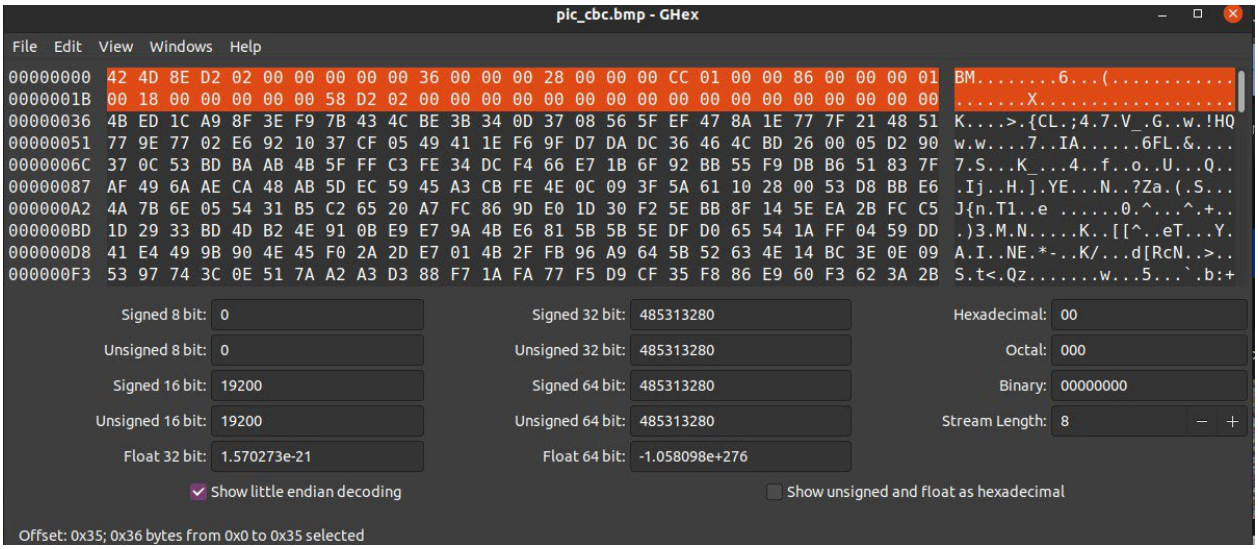


Figure 27: Replacing first 54 bytes with information header in pic_cbc.bmp using Ghex.

Q3.3

Below (Figure 28 and Figure 29) are the encrypted pictures.

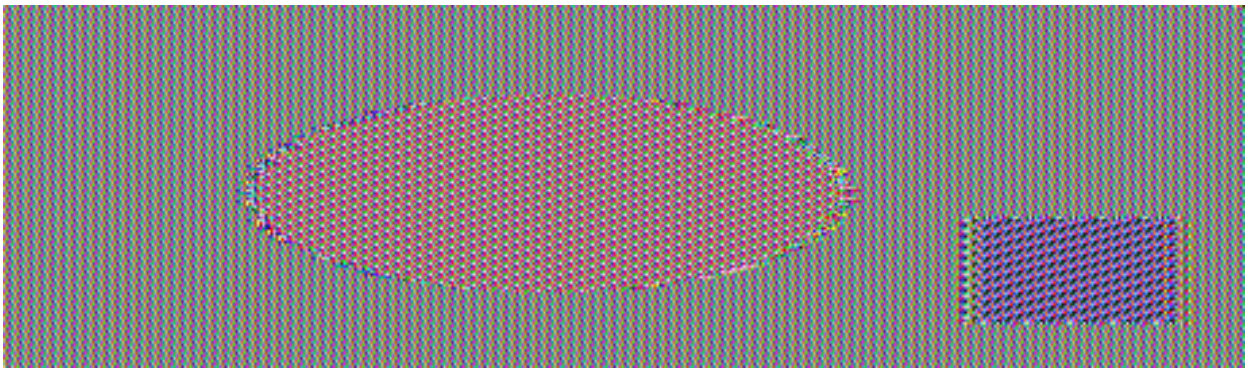


Figure 28: pic_ecb.bmp after setting information header correctly.

In Figure 28, with the image encrypted using AES ECB, the content and structure from the original pic_original.bmp can still be derived. Though the color had significantly changed, the big oval in the center and the rectangle to its right can still be observed. This is because AES ECB is deterministic. AES ECB divided the image into blocks and encrypted each block with the same key; therefore, if the blocks are identical, ECB will produce identical ciphered blocks. Because identical

block results in identical encrypted blocks, the pattern is preserved, which explains why the content of pic_original.bmp can still be observed.



Figure 29: pic_cbc.bmp after setting information header correctly.

In Figure 29, no useful information can be derived from the AES CBC encrypted picture. AES CBC uses a chaining mechanism where each plaintext block is combined with the previous ciphertext block using XOR operation before being encrypted by the cipher. For the first block, an Initialization Vector (IV) is used to start this process. Therefore, even if there are identical blocks, they will be mixed with a ciphertext from the previously encrypted block, resulting in no pattern preservation.

Section 4: Penetration Testing

Q4.1

To scan all host in the internal network, I used the command “nmap 192.168.60.0/24”. The result is shown in Figure 30. Nmap has successfully scan all the host in the internal subnet, including 192.168.60.5, 192.168.60.6, 192.168.60.7 and 192.168.60.11 (the router). Nmap showed the host status (“Host is up”) and open ports.

```
root@100e403f577f:/# nmap 192.168.60.0/24
Starting Nmap 7.80 ( https://nmap.org ) at 2026-08-26 02:14 UTC
Nmap scan report for 192.168.60.1 (192.168.60.1)
Host is up (0.0000080s latency).
Not shown: 997 closed ports
PORT   STATE SERVICE
21/tcp open  ftp
22/tcp open  ssh
23/tcp open  telnet

Nmap scan report for 192.168.60.5 (192.168.60.5)
Host is up (0.000014s latency).
Not shown: 999 closed ports
PORT   STATE SERVICE
23/tcp open  telnet

Nmap scan report for 192.168.60.6 (192.168.60.6)
Host is up (0.000013s latency).
Not shown: 999 closed ports
PORT   STATE SERVICE
23/tcp open  telnet

Nmap scan report for 192.168.60.7 (192.168.60.7)
Host is up (0.000013s latency).
Not shown: 999 closed ports
PORT   STATE SERVICE
23/tcp open  telnet

Nmap scan report for 192.168.60.11 (192.168.60.11)
Host is up (0.0000080s latency).
Not shown: 999 closed ports
PORT   STATE SERVICE
23/tcp open  telnet
```

Figure 30: Internal network scanned by Nmap.

Q4.2

To determine the Operating System (OS) of RMIT web server, nslookup command was used to obtain IP addresses from the server.

```
root@100e403f577f:/# nslookup rmit.edu.vn
Server:         127.0.0.11
Address:        127.0.0.11#53

Non-authoritative answer:
Name:   rmit.edu.vn
Address: 10.247.23.12
Name:   rmit.edu.vn
Address: 10.244.23.12
Name:   rmit.edu.vn
Address: 10.69.202.96
Name:   rmit.edu.vn
Address: 10.247.23.11
Name:   rmit.edu.vn
Address: 10.244.23.11
```

Figure 31: Using nslookup to obtain IP addresses from rmit.edu.vn.

Then, to determine RMIT web server OS, the command "nmap -O 10.247.23.12" was used, with the IP address taken from the previous step.

```
root@100e403f577f:/# nmap -O 10.247.23.12
Starting Nmap 7.80 ( https://nmap.org ) at 2026-08-26 02:33 UTC
Nmap scan report for sgwprddc02.RMIT.EDU.VN (10.247.23.12)
Host is up (0.024s latency).
Not shown: 988 filtered ports
PORT     STATE SERVICE
53/tcp   open  domain
88/tcp   open  kerberos-sec
135/tcp  open  msrpc
139/tcp  open  netbios-ssn
389/tcp  open  ldap
445/tcp  open  microsoft-ds
464/tcp  open  kpasswd5
593/tcp  open  http-rpc-epmap
636/tcp  open  ldapssl
3268/tcp open  globalcatLDAP
3269/tcp open  globalcatLDAPssl
3389/tcp open  ms-wbt-server
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Aggressive OS guesses: IBM AIX 5.3 (89%), IBM OS/2 Warp 2.0 (87%), Nintendo Wii game console (87%), IBM AIX 7.1 (86%), SMC SMC8014WG WAP (86%
), Huawei CloudEngine CE6800 switch (86%), D-Link DFL-700 firewall (86%), D-Link NetDefend firewall (85%), IBM AIX 4.3 (85%), HP PSC 2400-ser
ies Photosmart printer (85%)
No exact OS matches for host (test conditions non-ideal).

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.92 seconds
```

Figure 32: Using "nmap -O" to find server's OS.

In Figure 32, the result said "Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port" and "No exact OS matches for host (test conditions non-ideal)", which means that the OS scan was not successful due to firewall protection of RMIT web server. Nmap instead had given incorrect guesses on what the server's OS might be.

Q4.3

To obtain some port information and services running on RMIT web server, the command nmap -sV was used. The attempt is shown in Figure 33.

```
root@100e403f577f:/# nmap -sV 10.247.23.12
Starting Nmap 7.80 ( https://nmap.org ) at 2026-08-26 02:36 UTC
Nmap scan report for sgwprddc02.RMIT.EDU.VN (10.247.23.12)
Host is up (0.027s latency).
Not shown: 988 filtered ports
PORT     STATE SERVICE           VERSION
53/tcp   open  domain?
88/tcp   open  kerberos-sec      Microsoft Windows Kerberos (server time: 2026-08-27 03:01:48
135/tcp  open  msrpc?
139/tcp  open  netbios-ssn       Microsoft Windows netbios-ssn
389/tcp  open  ldap              Microsoft Windows Active Directory LDAP (Domain: RMIT.EDU.VN
445/tcp  open  microsoft-ds?
464/tcp  open  kpasswd5?
593/tcp  open  ncacn_http        Microsoft Windows RPC over HTTP 1.0
636/tcp  open  ssl/ldap          Microsoft Windows Active Directory LDAP (Domain: RMIT.EDU.VN
3268/tcp open  ldap              Microsoft Windows Active Directory LDAP (Domain: RMIT.EDU.VN
3269/tcp open  globalcatLDAPssl?
3389/tcp open  ms-wbt-server?
1 service unrecognized despite returning data. If you know the service/version, please submit
vice :
SF-Port53-TCP:V=7.80%I=7%D=8/26%Time=6A8E513C%P=x86_64-pc-linux-gnu%r(DNSV
SF:ersionBindReqTCP,20,"\0\x1e\0\x06\x81\x04\0\x01\0\0\0\0\0\0\x07version\
SF:x04bind\0\0\x10\0\x03")%r(DNSStatusRequestTCP,2B,"\0\)\xbe\[\x81\x82\0\
SF:x01\0\0\0\0\0\0\nzagent2655\x08su89-cdn\x03net\0\0\x01\0\x01")%r(Generi
SF:cLines,3C,"\0:>,\x81\x82\0\x01\0\0\0\0\0\0\x06_22224\x06_https\nzagent2
SF:655\x0bc6gj-static\x03net\0\0A\0\x01")%r(RTSPRequest,3B,"\x009\x19d\x81
SF:\x82\0\x01\0\0\0\0\0\0\x06_22224\x06_https\nzagent1611\nkbz0pwvxmv\x03c
SF:om\0\0A\0\x01")%r(Help,4B,"\0%A_\x81\x82\0\x01\0\0\0\0\0\0\nzagent2658\
SF:x04hola\x03org\0\0\x01\0\x01\x85`\x81\x82\0\x01\0\0\0\0\0\0\x02ad\x0bdo
SF:ubleclick\x03net\0\0\x01\0\x01")%r(LPDString,25,"\0#\xb6\x02\x81\x82\0\
SF:x01\0\0\0\0\0\0\x06static\x06hotjar\x03com\0\0A\0\x01")%r(LDAPBindReq,2
SF:F,"\0-g\x8e\x81\x82\0\x01\0\0\0\0\0\0\nzagent2658\x0cx-cdn-static\x03co
SF:m\0\0\x01\0\x01")%r(NCP,2B,"\0\)\x96\x80\x81\x82\0\x01\0\0\0\0\0\0\nzag
SF:ent2706\x08su89-cdn\x03net\0\0\x01\0\x01");
Service Info: Host: SGWPRDDC02; OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 125.54 seconds
```

Figure 33: Using "nmap -sV" to find ports and services running on server.

In Figure 33, it is observed that nmap had obtained much information about the port and services running on RMIT web server. Although it could not obtain any information on server default port (80 or 443), there was information on other ports running Windows' services. There was also a line saying: "OS: Windows", which helped to determine the answer to the question in Q4.2 that RMIT server's OS is Windows.

Q4.4

Figure 34 shows the command that was used to scan vulnerabilities in int_overflow_vuln.c using FlawFinder and the result.

```
[08/25/26]seed@VM:~$ flawfinder int_overflow_vuln.c
Flawfinder version 2.0.10, (C) 2001-2019 David A. Wheeler.
Number of rules (primarily dangerous function names) in C/C++ ruleset: 223
Examining int_overflow_vuln.c

FINAL RESULTS:

int_overflow_vuln.c:7:  [2] (buffer) char:
  Statically-sized arrays can be improperly restricted, leading to potential
  overflows or other issues (CWE-119!/CWE-120). Perform bounds checking, use
  functions that limit length, or ensure that the size is larger than the
  maximum possible length.
int_overflow_vuln.c:14:  [2] (buffer) memcpy:
  Does not check for buffer overflows when copying to destination (CWE-120).
  Make sure destination can always hold the source data.
int_overflow_vuln.c:15:  [2] (buffer) memcpy:
  Does not check for buffer overflows when copying to destination (CWE-120).
  Make sure destination can always hold the source data.
int_overflow_vuln.c:26:  [1] (buffer) strlen:
  Does not handle strings that are not \0-terminated; if given one it may
  perform an over-read (it could cause a crash if unprotected) (CWE-126).
int_overflow_vuln.c:27:  [1] (buffer) strlen:
  Does not handle strings that are not \0-terminated; if given one it may
  perform an over-read (it could cause a crash if unprotected) (CWE-126).

ANALYSIS SUMMARY:

Hits = 5
Lines analyzed = 32 in approximately 0.00 seconds (8383 lines/second)
Physical Source Lines of Code (SLOC) = 23
Hits@level = [0]   0 [1]   2 [2]   3 [3]   0 [4]   0 [5]   0
Hits@level+ = [0+]   5 [1+]   5 [2+]   3 [3+]   0 [4+]   0 [5+]   0
Hits/KSLOC@level+ = [0+] 217.391 [1+] 217.391 [2+] 130.435 [3+]   0 [4+]   0 [5+]   0
Minimum risk level = 1
Not every hit is necessarily a security vulnerability.
There may be other security vulnerabilities; review your code!
See 'Secure Programming HOWTO'
(https://dwheeler.com/secure-programs) for more information.
```

Figure 34: Using FlawFinder to scan vulnerabilities in int_overflow_vuln.c.

In the summary shown by FlawFinder, there were 5 detected errors (hits). In line 7 of int_overflow_vuln.c, it warned about potential overflow issue due to statically sized arrays. In line 14 and 15, it warned about function memcpy for it does not check for buffer overflow. In line 26 and 27, it warned about the function strlen may cause a buffer overread error if strings are not \0-terminated.

Q4.5

In line 7, 14 and 15, an extremely large number can be entered, causing the size calculation to overflow. The error then is passed to the function memcpy on line 14 and 15. In C language, the value \0 is used to mark the end of a string, since it has a

char data type, but no string data type[7]. Therefore, when function strlen is called, it starts from the first character of the memory location, continuously jumps to the next memory location to count and only stop when it sees character \0. But if a string does not have a \0 character, it won't stop at the boundary of that thread but will continue reading into adjacent memory areas, which is called buffer overread.

Q4.6

To fix the identified problem, in line 1, 14 and 15, the memcpy function should be updated so that it can calculate the length of the parameters, check whether it has exceeded the limit of the data type before passing them into the memcpy function. In line 26 and 27, the function strlen should be updated to be able to check if the passed string has a \0 terminating character before measuring its length.

Table of Figures

Figure 1: Building and starting containers.....7
Figure 2: All containers and their IDs.....8
Figure 3: Snort rule set in .etc.snort/rules/local.rules 9
Figure 4: Ping subnet's broadcast IP address from 10.9.0.5. 10
Figure 5: Snort alerts seen in router's console in Q1.8. 10
Figure 6: Ping IP1 from internal host 192.168.60.5 11
Figure 7: Snort alerts seen in router's console in Q1.8. 11
Figure 8: Result of firewall rule in Q2.1.....12
Figure 9: Result of firewall rule in Q2.2.....13
Figure 10: Firewall rules set up for Q2.3.....13
Figure 11: Telnet from 192.168.0.5 to 10.9.0.5 in Q2.3. 14
Figure 12: Telnet from 10.9.0.5 to host 192.168.60.5 in Q2.3. 14
Figure 13: Telnet from 10.9.0.5 to host 192.168.60.6 in Q2.3. 14
Figure 14: Firewall rules set up for Q2.4.....15
Figure 15: Telnet from 192.168.0.5 to 10.9.0.5 in Q2.4. 15
Figure 16: Telnet from 10.9.0.5 to host 192.168.60.5 in Q2.4. 16
Figure 17: Telnet from 10.9.0.5 to host 192.168.60.6 in Q2.4. 16
Figure 18: Firewall rules set up for Q2.4.....16
Figure 19: Telnet from 192.168.0.5 to 10.9.0.5 in Q2.5. 17
Figure 20: Telnet from 10.9.0.5 to host 192.168.60.5 in Q2.5. 17
Figure 21: Telnet from 10.9.0.5 to host 192.168.60.6 in Q2.5. 17
Figure 22: Encrypting pic_original.bmp using AES ECB 18
Figure 23: Encrypting pic_original.bmp using AES CBC.....18
Figure 24: Encrypted pictures.....18
Figure 25: Extracting header information from pic_original.bmp using GHex. 19
Figure 26: Replacing first 54 bytes with information header in pic_ecb.bmp using Ghex. 19
Figure 27: Replacing first 54 bytes with information header in pic_cbc.bmp using Ghex. 20
Figure 28: pic_ecb.bmp after setting information header correctly.....20
Figure 29: pic_cbc.bmp after setting information header correctly.....21
Figure 30: Internal network scanned by Nmap.....22
Figure 31: Using nslookup to obtain IP addresses from rmit.edu.vn.....23

Figure 32: Using “nmap -O” to find server’s OS. 23
Figure 33: Using “nmap -sV” to find ports and services running on server.....24
Figure 34: Using FlawFinder to scan vulnerabilities in int_overflow_vuln.c 25

Reference

[1] “How to Install and Configure Snort on Ubuntu.” Accessed: Aug. 25, 2026. [Online]. Available: https://letsdefend.io/blog/how-to-install-and-configure-snort-on-ubuntu

[2] “Snort 3 Rule Writing Guide - Snort 3 Rule Writing Guide.” Accessed: Aug. 25, 2026. [Online]. Available: https://docs.snort.org/welcome

[3] “What is ICMP Ping? How It Works + Troubleshooting Guide.” Accessed: Aug. 25, 2026. [Online]. Available: https://www.dotcom-monitor.com/learn/glossary/icmp-ping/

[4] Snort Team, “SNORT Users Manual,” Apr. 08, 2020. Accessed: Aug. 25, 2026. [Online]. Available: https://snort-org-site.s3.amazonaws.com/production/document_files/files/000/000/249/original/snort_manual.pdf?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIAU7AK5ITMBDFJ4Z4P%2F20260825%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Date=20260825T074524Z&X-Amz-Expires=172800&X-Amz-SignedHeaders=host&X-Amz-Signature=f0290a75490f2a746694ae3c38bd52a2ff4782c01edd94ed4f6230c0c7fb6323

[5] “iptables(8) - Linux manual page.” Accessed: Aug. 26, 2026. [Online]. Available: https://man7.org/linux/man-pages/man8/iptables.8.html

[6] “[Network] TÌM HIỂU VỀ IPTABLES.” Accessed: Aug. 26, 2026. [Online]. Available: https://viblo.asia/p/network-tim-hieu-ve-iptables-n7prv348RKod

[7] “C++ - ‘Kết thúc bằng số 0’ nghĩa là gì? - Stack Overflow.” Accessed: Aug. 29, 2026. [Online]. Available: https://stackoverflow.com/questions/2667648/what-does-it-mean-to-be-terminated-by-a-zero